

Byte 1: The front door of the application

File: Level3Application.java

Builds on: None — starting point

In plain terms:

This is the single entry point that boots the whole Java backend. Its annotations also switch on three background capabilities the app needs: message listening, retry logic, and scheduled tasks.

The code:

```
@SpringBootApplication
@EnableRabbit
@EnableRetry
@EnableScheduling
public class Level3Application {
    public static void main(String[] args) {
        SpringApplication.run(Level3Application.class, args);
    }
}
```

What's happening:

`@SpringBootApplication` tells Spring Boot to scan the project, wire up dependencies automatically, and start a web server. `@EnableRabbit` activates RabbitMQ message listeners, `@EnableRetry` allows automatic retries of failed operations, and `@EnableScheduling` allows methods to run on a timer. `main` just launches everything.

Why it matters:

Without this file nothing runs. The three `@Enable...` annotations are easy to miss but the sync listener, retry interceptor, and polling scheduler silently do nothing if their annotation is absent.

Byte 2: One file that configures every connection

File: application.properties

Builds on: Byte 1 (this is what the booted app reads)

In plain terms:

This file is the control panel for the backend. It says where PostgreSQL, MongoDB, RabbitMQ, and Elasticsearch live, which queue and index to use, and how JSON should look — with environment variables overriding the defaults inside Docker.

The code:

```
spring.datasource.url=${POSTGRES_URL:jdbc:postgresql://localhost:5445/ecom_level3}
spring.data.mongodb.uri=${MONGO_URL:mongodb://localhost:27018/ecom_level3}
spring.rabbitmq.host=${RABBITMQ_HOST:localhost}
spring.rabbitmq.port=${RABBITMQ_PORT:5683}
app.elasticsearch.url=${ELASTICSEARCH_URL:http://localhost:9211}
```

```
app.elasticsearch.index=${ELASTICSEARCH_INDEX:level3_orders}
app.sync.queue=${SYNC_QUEUE:level3_order_sync}
spring.jackson.property-naming-strategy=SNAKE_CASE
```

What's happening:

Each line maps a setting to an environment variable with a fallback after the colon. `SNAKE\_CASE` makes Java's `orderId` serialize as `order\_id`, matching the Level 2 Python API so the same frontend works unchanged.

Why it matters:

Level 3 isolation lives here: database name, queue, index, and ports are all Level 3—specific. Point this file at Level 2 values and you would corrupt Level 2 data — that is why every value is namespaced.

Byte 3: The ingredient list for the build

File: pom.xml

Builds on: Byte 1 (declares what the app is built from)

In plain terms:

This is Maven's shopping list. It declares Spring Boot 3.2.5 on Java 17 plus one starter dependency per infrastructure piece: web, database, catalog, messaging, validation, retry, and API docs.

The code:

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>3.2.5</version>
</parent>
<dependencies>
  <dependency><artifactId>spring-boot-starter-web</artifactId></dependency>
  <dependency><artifactId>spring-boot-starter-data-jpa</artifactId></dependency>
  <dependency><artifactId>spring-boot-starter-data-mongodb</artifactId></dependency>
  <dependency><artifactId>spring-boot-starter-amqp</artifactId></dependency>
  <dependency><artifactId>spring-boot-starter-validation</artifactId></dependency>
  <dependency><artifactId>spring-retry</artifactId></dependency>
  <dependency><artifactId>postgresql</artifactId><scope>runtime</scope></dependency>
</dependencies>
```

What's happening:

The parent supplies compatible versions for everything, so only names are needed. Each "starter" pulls in a whole capability (web server, database access, message queue). Notably absent: any Elasticsearch client library — the app talks to Elasticsearch over plain HTTP instead (see Bytes 19–20).

Why it matters:

This file is why there is no version-conflict pain. Test-only extras (H2 database, test starter) use ``<scope>test</scope>`` so they never ship in the Docker image.

Byte 4: RabbitMQ road layout — queue, exchange, binding

File: RabbitMqConfig.java

Builds on: Byte 2 (reads `app.sync.queue` / `app.sync.exchange`)

In plain terms:

RabbitMQ is a post office for sync messages. This byte creates a dedicated mailbox (queue), a sorting office (exchange), and the rule connecting them (binding) — all with Level 3-only names.

The code:

```
@Bean
public Queue orderSyncQueue() {
    return QueueBuilder.durable(syncQueue).build();    // "level3_order_sync"
}

@Bean
public DirectExchange orderSyncExchange() {
    return new DirectExchange(syncExchange, true, false);    // "level3_orders"
}

@Bean
public Binding orderSyncBinding(Queue orderSyncQueue, DirectExchange orderSyncExchange) {
    return BindingBuilder.bind(orderSyncQueue).to(orderSyncExchange).with(syncQueue);
}
```

What's happening:

`@Bean` methods hand objects to Spring to manage (this is dependency injection: you declare **what** you need, Spring builds and passes it around). The queue is durable so messages survive a broker restart. The binding says "anything sent to this exchange with this routing key lands in this queue."

Why it matters:

These names are the isolation boundary for messaging. A typo here would publish sync events into Level 2's queue or listen on it — the rest of the system assumes these exact names.

Byte 5: Making the listener resilient with bounded retries

File: RabbitMqConfig.java

Builds on: Byte 4 (customizes how the queue from Byte 4 is consumed)

In plain terms:

If Elasticsearch is down when a sync message arrives, the listener should wait and try again a few times — not crash, and not silently drop the order. This byte builds that patience into the listener.

The code:

```
@Bean
public SimpleRabbitListenerContainerFactory syncListenerFactory(
```

```

    ConnectionFactory connectionFactory, Jackson2JsonMessageConverter converter) {
SimpleRabbitListenerContainerFactory factory = new SimpleRabbitListenerContainerFactory();
factory.setConnectionFactory(connectionFactory);
factory.setMessageConverter(converter);
factory.setDefaultRequeueRejected(false);
RetryOperationsInterceptor retry = RetryInterceptorBuilder.stateless()
    .maxAttempts(Math.max(1, maxRetries))        // 5 attempts
    .backOffOptions(8000, 2.0, 60000)           // 8s, doubling, max 60s
    .build();
factory.setAdviceChain(retry);
return factory;
}

```

What's happening:

A "listener container factory" is a template for message-listening workers; the sync listener (Byte 34) explicitly asks for this one by name. The retry interceptor wraps each delivery: on failure it waits 8s, then 16s, and so on, up to 5 tries. `setDefaultRequeueRejected(false)` means a permanently failing message is finally discarded, not looped forever.

Why it matters:

This is the "Elasticsearch down" survival story for the event path. Without it, one ES outage would either lose syncs or poison the queue. The polling scheduler (Byte 36) is the second safety net.

Byte 6: The shared HTTP caller for Elasticsearch

File: RestTemplateConfig.java

Builds on: Byte 1 (a shared helper bean)

In plain terms:

The app needs to make plain HTTP calls to Elasticsearch. This file creates one shared, pre-tuned HTTP client (RestTemplate) that the search code reuses instead of building a new connection each time.

The code:

```

@Bean
public RestTemplate restTemplate() {
    SimpleClientHttpRequestFactory factory = new SimpleClientHttpRequestFactory();
    factory.setConnectTimeout(10000);
    factory.setReadTimeout(15000);
    return new RestTemplate(factory);
}

```

What's happening:

`RestTemplate` is Spring's HTTP client. The timeouts say "give up connecting after 10s and waiting for a reply after 15s" so a hung Elasticsearch cannot freeze the whole API forever.

Why it matters:

Every Elasticsearch operation funnels through this one bean, so timeout behavior is consistent. It is also what tests mock to simulate Elasticsearch without a real server.

Byte 7: Letting the browser talk to the API

File: WebConfig.java

Builds on: Byte 1 (web-layer setup)

In plain terms:

Browsers block web pages from calling APIs on other addresses (CORS policy). This file explicitly permits the known frontend addresses — local dev server, nginx, and plain localhost — to call the Java API.

The code:

```
registry.addMapping("/**")
    .allowedOrigins("http://localhost:5173", "http://127.0.0.1:5173",
        "http://localhost:8080", "http://127.0.0.1:8080",
        "http://localhost", "http://127.0.0.1",
        "http://localhost:8082", "http://127.0.0.1:8082")
    .allowedMethods("*")
    .allowedHeaders("*")
    .allowCredentials(true);
```

What's happening:

`/\*\*` means "all API paths". The list names every origin the frontend might be served from: the Vite dev server (5173), the Level 2/3 nginx ports, and bare localhost. In Docker the browser actually uses same-origin `/api` through nginx, so this mostly matters for local development.

Why it matters:

Without this, the frontend's API calls fail in the browser with cryptic CORS errors even though the API itself works fine. If you serve the UI from a new port, it must be added here.

Byte 8: The customer record

File: Level3User.java

Builds on: None — first domain model (JPA entity = a Java class mapped to a database table)

In plain terms:

A customer is just an id, a name, and a unique email. The table is named `level3\_users` so it can never collide with Level 2's `users` table.

The code:

```
@Entity
@Table(name = "level3_users")
public class Level3User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```

private Long id;

@Column(nullable = false, length = 120)
private String name;

@Column(nullable = false, unique = true, length = 255)
private String email;
}

```

What's happening:

JPA (Java Persistence API) with Hibernate translates classes to tables: `@Entity` marks it persistent, `@Table` fixes the exact table name, `@Id` + `@GeneratedValue` means "database auto-numbers the rows." `unique = true` on email is enforced by the database itself.

Why it matters:

There is deliberately no login or password here — users are seeded demo identities the frontend picks from a dropdown. The `level3\_` table prefix is the PostgreSQL isolation mechanism.

Byte 9: The order record

File: Level3Order.java

Builds on: Byte 8 (an order belongs to a user)

In plain terms:

An order links to its customer, carries a status, a total, and two automatic timestamps: when it was placed and when it was last touched. The "last touched" timestamp is what the polling sync uses to find changed orders.

The code:

```

@Entity
@Table(name = "level3_orders")
public class Level3Order {
    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "user_id", nullable = false)
    private Level3User user;

    @CreationTimestamp
    @Column(name = "order_date", nullable = false, updatable = false)
    private Instant orderDate;

    @Column(nullable = false, length = 32)
    private String status = "PENDING";

    @UpdateTimestamp
    @Column(name = "updated_at", nullable = false)
    private Instant updatedAt;

    @OneToMany(mappedBy = "order", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<Level3OrderItem> items = new ArrayList<>();
}

```

```
}
```

What's happening:

`@ManyToOne` is the foreign key to the user; `LAZY` means the user row is only loaded when actually read.

`@CreationTimestamp`/`@UpdateTimestamp` let Hibernate stamp the dates automatically. The `items` list with

`cascade = ALL` means saving an order automatically saves its line items — one call persists the whole order.

Why it matters:

`updated\_at` drives Strategy B polling, and the cascade makes order creation atomic: order plus items commit together or roll back together, so partial orders can never exist.

Byte 10: The snapshot line item — history insurance

File: Level3OrderItem.java

Builds on: Byte 9 (items belong to an order)

In plain terms:

Each line item stores the product id plus a frozen copy of the title and price from the moment of purchase. If the catalog later renames "Wireless Mouse" or raises its price, old receipts don't change.

The code:

```
@Entity
@Table(name = "level3_order_items")
public class Level3OrderItem {
    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "order_id", nullable = false)
    private Level3Order order;

    /** MongoDB product id as string (no cross-database FK possible). */
    @Column(name = "product_id", nullable = false, length = 64)
    private String productId;

    @Column(nullable = false, length = 255)
    private String title = "";

    @Column(nullable = false)
    private Integer quantity;

    @Column(name = "unit_price", nullable = false, precision = 10, scale = 2)
    private BigDecimal unitPrice;
}
```

What's happening:

`product\_id` is deliberately a plain string, not a foreign key — PostgreSQL cannot enforce a key into MongoDB, so the comment documents that. Money uses `BigDecimal` (exact decimal arithmetic) instead of `double` (approximate binary floating point) to avoid rounding errors in totals.

Why it matters:

This snapshot is the reason the search index and order history are built from PostgreSQL only and never consult MongoDB. It is the single most important data-model decision in the project.

Byte 11: The catalog document in MongoDB

File: Product.java

Builds on: Byte 10 (products are what gets snapshotted at checkout)

In plain terms:

Products live in MongoDB as flexible documents, not rigid table rows — each can have different attributes (a keyboard has a "switch", a monitor has a "size") and a list of variants, without schema migrations.

The code:

```
@Document(collection = "products")
public class Product {
    @Id
    private String id;           // MongoDB ObjectId as string

    @Indexed(unique = true)
    private String sku;

    private String title;
    private double price;
    private String category;
    private List<String> tags = new ArrayList<>();
    private Map<String, Object> attributes = new HashMap<>();
    private List<Variant> variants = new ArrayList<>();
    private boolean active = true;

    @Field("updated_at")
    private Instant updatedAt;

    public static class Variant {
        private String name = "";
        private String sku = "";
        private double priceAdjustment = 0.0;
    }
}
```

What's happening:

`@Document` maps the class to the `products` collection inside the `ecom\_level3` database (database chosen in Byte 2). `attributes` is a free-form map — the "schemaless" part. `active = false` hides discontinued products from the storefront while keeping them orderable-history intact. `Variant` is a nested sub-document (e.g. Black vs White mouse).

Why it matters:

This is the write-path discipline made concrete: catalog writes go here and **only** here. Orders never write to MongoDB, so there is exactly one owner of product data.

Byte 12: Finding orders in PostgreSQL

File: OrderRepository.java

Builds on: Byte 9 (queries over the order entity)

In plain terms:

A repository is a ready-made data-access object: you declare method names and Spring writes the SQL. This one adds order-specific lookups — fetch an order with everything attached, filter by customer/status, and find recently changed orders for the poller.

The code:

```
public interface OrderRepository extends JpaRepository<Level3Order, Long> {
    List<Level3Order> findByUpdatedAtGreaterThanOrEqualToOrderByDesc(Instant cutoff, Pageable pageable);

    @Query("select o from Level3Order o left join fetch o.items left join fetch o.user where o.id = :id")
    java.util.Optional<Level3Order> findByIdWithItems(Long id);

    Page<Level3Order> findByUser_Id(Long userId, Pageable pageable);
    Page<Level3Order> findByUser_IdAndStatus(Long userId, String status, Pageable pageable);
}
```

What's happening:

Long method names are parsed into queries (`findBy...GreaterThanOrEqualTo...OrderBy...Desc`). The `@Query` one eagerly loads items and user in a single query to avoid lazy-loading failures later. `findByUser\_Id` (with underscore) walks the `user` association to its `id` — plain `findById` would fail because no such field exists.

Why it matters:

The poller (Byte 29) and every order read depend on these. The underscore naming quirk in Byte 71's gotcha bit a real test run, so it is worth memorizing.

Byte 13: Finding products in MongoDB

File: ProductRepository.java

Builds on: Byte 11 (queries over the product document)

In plain terms:

The Mongo counterpart of Byte 12: storefront listing (active only), batch lookup for checkout validation, and SKU lookup for duplicate detection.

The code:

```
public interface ProductRepository extends MongoRepository<Product, String> {
    List<Product> findByActiveTrue();
    List<Product> findByIdIn(List<String> ids);
    Optional<Product> findBySku(String sku);
}
```

What's happening:

`MongoRepository` is the MongoDB flavor of the same repository idea — identical style, different database. `findByActiveTrue` powers the storefront filter; `findByIdIn` grabs all cart products in one round trip during order creation.

Why it matters:

Checkout calls `findByIdIn` once per order rather than once per line item. Missing ids simply come back absent, which is how "product not found" (404) is detected.

Byte 14: The pass-through user lookup

File: UserRepository.java

Builds on: Byte 8 (queries over the user entity)

In plain terms:

The simplest repository in the project: full user list plus email lookup, with zero custom code — Spring implements it from the interface alone.

The code:

```
public interface UserRepository extends JpaRepository<Level3User, Long> {  
    Optional<Level3User> findByEmail(String email);  
}
```

What's happening:

Inheriting `JpaRepository` already provides `findAll`, `findById`, `save`, `count`, and delete operations. The single declared method adds an email search used by seed verification.

Why it matters:

It shows the baseline: you only write repository code when the built-in operations aren't enough (contrast Bytes 12–13).

Byte 15: The pass-through order-item store

File: OrderItemRepository.java

Builds on: Byte 10 (persists the snapshot lines)

In plain terms:

An empty-looking repository that exists so the seed routine can wipe and count line items directly. Order creation itself saves items through the order's cascade (Byte 9), not through here.

The code:

```
public interface OrderItemRepository extends JpaRepository<Level3OrderItem, Long> {  
}
```

What's happening:

No methods at all — every operation (`deleteAll`, `count`, `save`) is inherited. It is injected into the seed service for cleanup and quota checks.

Why it matters:

Its emptiness is intentional and informative: if you find yourself adding query methods here for order reads, you are probably bypassing the order aggregate and risking inconsistent reads.

Byte 16: The shapes of the API — DTOs

File: Dtos.java

Builds on: Bytes 8–11 (DTOs mirror the models for the outside world)

In plain terms:

DTOs (Data Transfer Objects) are the exact request/response shapes the API speaks. They use Java records — compact immutable value holders — with validation annotations, and Jackson converts them to the snake_case JSON the Vue frontend expects.

The code:

```
public record OrderCreate(
    @NotNull Long userId,
    @NotNull @Size(min = 1) List<@Valid OrderItemIn> items) {}

public record OrderItemIn(
    @NotBlank String productId,
    @Min(1) @Max(1000) int quantity) {}

public record OrderSearchRequest(
    String q, List<String> statuses,
    String dateFrom, String dateTo,
    Double minTotal, Double maxTotal,
    String sort, Integer page, Integer size) {}
```

What's happening:

Annotations like `@NotNull`, `@Size(min = 1)`, `@Min/@Max` declare the rules; Spring checks them automatically before controller code runs, and failures become 422s (Byte 18). `@Valid` on the list elements cascades validation into each line item. Because of the global SNAKE_CASE setting, `userId` arrives as `user\_id` with no extra annotations.

Why it matters:

Every compatibility promise to the frontend lives here: field names, required fields, quantity limits, and search filter shapes. Change a record and you change the API contract.

Byte 17: Typed errors instead of generic exceptions

File: NotFoundException.java

Builds on: Byte 16 (thrown when DTO-driven operations fail to find things)

In plain terms:

Three tiny exception classes give failures precise meanings — "not found" (404), "conflict" (409, e.g. duplicate SKU), and "unprocessable" (422, e.g. bad status) — each carrying its own HTTP status on the class itself.

The code:

```
@ResponseStatus(HttpStatus.NOT_FOUND)
public class NotFoundException extends RuntimeException {
    public NotFoundException(String message) { super(message); }
}

/** Maps to 422 to match Level 2 FastAPI validation behaviour. */
@ResponseStatus(HttpStatus.UNPROCESSABLE_ENTITY)
public class UnprocessableException extends RuntimeException {
    public UnprocessableException(String message) { super(message); }
}
// ConflictException follows the same pattern with HttpStatus.CONFLICT.
```

What's happening:

`@ResponseStatus` tells Spring which HTTP code to send when the exception escapes a controller. Service code throws domain-meaningful errors (`throw new NotFoundException("User not found")`) without importing any web classes.

Why it matters:

The 422 mapping is a deliberate Level 2 compatibility choice — Spring would otherwise return 400 for validation problems while FastAPI returns 422, and the frontend distinguishes them.

Byte 18: Translating failures into frontend-friendly errors

File: GlobalExceptionHandler.java

Builds on: Bytes 16–17 (handles validation and typed-exception failures)

In plain terms:

This is the single place that converts Java-side failures into the `{detail: message}` JSON errors the frontend already knows how to display. Bean-validation failures are normalized to 422 here to match Level 2.

The code:

```
@RestControllerAdvice
public class GlobalExceptionHandler {
    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<Map<String, Object>> handleValidation(MethodArgumentNotValidException ex) {
        String msg = ex.getBindingResult().getFieldErrors().stream()
            .map(f -> f.getField() + ": " + f.getDefaultMessage())
            .findFirst().orElse("Validation failed");
        return ResponseEntity.status(HttpStatus.UNPROCESSABLE_ENTITY).body(Map.of("detail", msg));
    }
}
```

```
// ...plus handlers for ConstraintViolation, unreadable bodies,
// type mismatches (all 422) and the typed exceptions from Byte 17.
}
```

What's happening:

`@RestControllerAdvice` makes the class a global catch-all for controller errors. Each `@ExceptionHandler` method handles one failure family and returns the same `{detail}` envelope with the appropriate status.

Why it matters:

Frontend error display (`e.response?.data?.detail` in Checkout.vue and CatalogAdmin.vue) depends on this envelope. Without it, clients would get Spring's default HTML-heavy error pages.

Byte 19: Teaching Elasticsearch what an order looks like

File: ElasticOrderIndex.java

Builds on: Byte 6 (uses the shared RestTemplate) and Byte 10 (indexes snapshot data)

In plain terms:

Elasticsearch is the fast text-search copy of orders used only by admin search. This first half of the file defines the index structure (which fields are searchable text vs exact keywords vs numbers) and can create or delete the index.

The code:

```
private static final Map<String, Object> ORDER_INDEX_MAPPING = Map.of(
    "mappings", Map.of("properties", Map.of(
        "order_id", Map.of("type", "integer"),
        "status", Map.of("type", "keyword"),
        "total_amount", Map.of("type", "float"),
        "customer", Map.of("properties", Map.of(
            "name", Map.of("type", "text", "fields",
                Map.of("keyword", Map.of("type", "keyword")))),
        "items", Map.of("type", "nested", "properties", Map.of(
            "title", Map.of("type", "text"))))));

public void ensureIndex() {
    if (indexExists()) return;
    restTemplate.exchange(esUrl + "/" + index, HttpMethod.PUT,
        new HttpEntity<>(ORDER_INDEX_MAPPING, headers), Map.class);
}
```

What's happening:

The mapping declares field types: `text` for full-text search (customer names, item titles), `keyword` for exact filters (status), numbers/dates for ranges and sorting. Items are `nested` so each line item is searched as its own unit. `ensureIndex` creates the index only if missing — safe to call on every sync.

Why it matters:

Get the mapping wrong and admin search silently returns bad results (e.g. analyzing a status field would break exact filtering). `ensureIndex` makes every writer self-healing: a deleted index is recreated on next use.

Byte 20: Reading and writing search documents over HTTP

File: ElasticOrderIndex.java

Builds on: Byte 19 (operates on the index defined there)

In plain terms:

This second half is the actual document API: save one order, save many at once, fetch one back for the sync-status check, and run a search. Every call targets the isolated `level3\_orders` index.

The code:

```
/** Idempotent upsert — document id = PostgreSQL order id. */
public void indexDocument(long orderId, Map<String, Object> document) {
    restTemplate.exchange(esUrl + "/" + index + "/_doc/" + orderId,
        HttpMethod.PUT, new HttpEntity<>(document, headers), Map.class);
}

public void bulkIndex(Map<Long, Map<String, Object>> documents) {
    // builds newline-delimited JSON: one {"index": {"_id": "42"}} line
    // followed by the document line, per order, POSTed to /_bulk
}

public Map<String, Object> search(Map<String, Object> body) { ... }
public Map<String, Object> getDocument(long orderId) { ... } // 404 -> null
```

What's happening:

`PUT /\_doc/{orderId}` overwrites-or-creates the document with that id — repeating it is harmless, which is what "idempotent" means here. The bulk variant packs many such writes into one HTTP call using Elasticsearch's NDJSON format. A 404 on fetch returns `null` (order not yet synced) instead of throwing; anything else becomes an "Elasticsearch unavailable" error that triggers retries.

Why it matters:

Using document id = PostgreSQL order id is the idempotency backbone: dual-write, polling, and reindex can all overlap without creating duplicates.

Byte 21: The three magic words for order status

File: OrderStatuses.java

Builds on: Byte 17 (invalid values become 422 through here)

In plain terms:

Orders may only ever be PENDING, PROCESSING, or SHIPPED — always uppercase. This tiny utility normalizes user input ("shipped" → "SHIPPED") and rejects anything else with a clear error.

The code:

```
public static final List<String> ALL = List.of("PENDING", "PROCESSING", "SHIPPED");

public static String requireValid(String value) {
    String norm = normalize(value);    // trim + UPPERCASE
    if (!SET.contains(norm))
        throw new UnprocessableException("status must be one of " + ALL);
    return norm;
}
```

What's happening:

A static helper (no Spring wiring needed) used by order updates, order listing filters, and search validation alike. `requireValidList` does the same for the multi-status search filter.

Why it matters:

One canonical list means the API, search, seed data, and frontend dropdowns can never disagree about valid statuses. It also keeps Elasticsearch's `keyword` status field clean.

Byte 22: Creating an order — validate, snapshot, commit

File: OrderService.java

Builds on: Bytes 10, 12–13, 16, 21 (snapshot model, lookups, DTOs, statuses)

In plain terms:

This is the money method: it checks the customer exists, checks every cart product exists and is still sold, freezes each product's title and price into the order, and saves order plus items atomically. Anything invalid aborts before anything is saved.

The code:

```
@Transactional
public Level3Order createOrder(OrderCreate payload) {
    // 1. reject duplicate product_ids in the cart -> 422
    Level3User user = users.findById(payload.userId())
        .orElseThrow(() -> new NotFoundException("User not found"));
    Map<String, Product> found = products.lookupByIds(ids);
    // 2. missing ids -> 404 ; inactive products -> 422
    Level3Order order = new Level3Order();
    order.setUser(user); order.setStatus("PENDING");
    for (OrderItemIn in : payload.items()) {
        Product p = found.get(in.productId());
        item.setTitle(p.getTitle());           // snapshot, not a reference
        item.setUnitPrice(...p.getPrice()); // snapshot, not a reference
        total = total.add(unit.multiply(...quantity...));
    }
    order.setTotalAmount(total); return orders.save(order); // cascade saves items
}
```

What's happening:

`@Transactional` makes the whole method one database transaction: any exception rolls everything back, so failed checkouts leave no partial rows. Crucially, the method only **returns** the saved order — it does NOT publish the sync event. The controller publishes after this method completes, which guarantees the publish happens only after commit (Byte 40).

Why it matters:

Order of operations is the correctness story: validate MongoDB, commit PostgreSQL, **then** announce. Publishing inside the transaction could announce an order that later rolls back.

Byte 23: Reading orders and flattening them for the outside world

File: OrderService.java

Builds on: Byte 22 (reads what creation wrote)

In plain terms:

The companion half of the service: fetch orders (single, paged, filtered), change status, and convert database rows into API responses and search-index documents — the latter built strictly from PostgreSQL, never touching MongoDB.

The code:

```
public static Map<String, Object> toOrderDocument(Level30Order order) {
    doc.put("order_id", order.getId());
    doc.put("status", order.getStatus());
    doc.put("total_amount", order.getTotalAmount().doubleValue());
    doc.put("customer", Map.of("id", ..., "name", name, "email", email));
    doc.put("items", items.stream().map(it -> Map.of(
        "product_id", it.getProductId(), "title", it.getTitle(),
        "quantity", it.getQuantity(), "unit_price", ...)).toList());
}

public static OrderOut toOrderOut(Level30Order order) { /* same data, API shape + line_total */ }
```

What's happening:

Both converters are pure static functions of an already-loaded order — no database or Mongo calls inside. `toOrderDocument` produces exactly the Elasticsearch shape from Byte 19's mapping; `toOrderOut` adds per-line `line\_total` for receipts.

Why it matters:

Because the search document is derived from the frozen PostgreSQL snapshot, a later catalog rename can never rewrite history — and the seed's snapshot test (Byte 32) proves it on every fresh install.

Byte 24: The catalog's gatekeeper

File: ProductService.java

Builds on: Bytes 11, 13, 16 (product model, repo, DTOs)

In plain terms:

All MongoDB product access flows through here: list (storefront vs admin), fetch one, create, and update — with duplicate-SKU protection that returns a 409 conflict.

The code:

```
public List<ProductOut> listProducts(boolean all) {
    List<Product> docs = all ? products.findAll() : products.findByActiveTrue();
    return docs.stream().map(ProductService::toOut).toList();
}

public ProductOut createProduct(ProductIn in) {
    products.findBySku(in.sku()).ifPresent(p -> {
        throw new ConflictException("sku already exists: " + in.sku());
    });
    ... // apply fields, stamp updatedAt, save (duplicate-key race -> 409 too)
}
```

What's happening:

The `all` flag is the storefront-vs-admin switch: shoppers see only active products, the catalog manager sees everything including discontinued ones. Updates patch only catalog fields and stamp `updated\_at`; they never touch orders or the search index.

Why it matters:

Centralizing catalog access in one service is what makes "MongoDB owns the catalog" enforceable — no controller reaches past this class into the repository.

Byte 25: Turning UI filters into an Elasticsearch query

File: SearchService.java

Builds on: Byte 19 (targets the mapping defined there)

In plain terms:

Admin search lets staff filter by text, status, date range, and total range. This method translates those friendly filters into the nested boolean query language Elasticsearch understands.

The code:

```
public static Map<String, Object> buildEsQuery(String q, List<String> statuses,
    String dateFrom, String dateTo, Double minTotal, Double maxTotal) {
    // text "Wireless" -> multi_match over customer.name^3, customer.email^2, items.title^2
    // "42" or "ORD-42" -> exact term filter on order_id
    // statuses -> term (one) or terms (many) filter on status
    // dates/totals -> range filters
    // nothing set -> {"match_all": {}}
}
```

What's happening:

Free text becomes a `multi_match` query with boosted fields (name matches rank higher than item matches). Pure numbers (or "ORD-42") shortcut to an exact order-id lookup. Everything else becomes `filter` clauses, which match exactly without affecting relevance scoring.

Why it matters:

This is the "why not SQL search" answer made concrete: one structured query does full-text, exact, range, and id search together — the kind of query that gets expensive as `LIKE '%...%'` joins in PostgreSQL.

Byte 26: Running the search and shaping the answer

File: SearchService.java

Builds on: Byte 25 (executes the query built there)

In plain terms:

This method sends the query with sorting and pagination, then repackages Elasticsearch's raw reply into the `{total, page, size, results, aggs}` envelope the admin screen renders — including revenue and per-status counts.

The code:

```
body.put("sort", SORT_MAP.getOrDefault(sort, SORT_MAP.get("newest")));
body.put("from", (page - 1) * size); body.put("size", size);
body.put("aggs", Map.of(
    "total_revenue", Map.of("sum", Map.of("field", "total_amount")),
    "by_status", Map.of("terms", Map.of("field", "status"))));
// ...then unpack hits + aggregations into {total, page, size, results, aggs}
```

What's happening:

`SORT_MAP` translates the four UI sort choices (newest/oldest/total asc/desc) into Elasticsearch sort clauses. Each hit's `_source` document is copied out with its relevance `score` attached. Aggregations are computed server-side so the revenue KPI needs no extra queries.

Why it matters:

The frontend's KPI cards (filtered revenue, per-status counts, total hits) all come from this one response — a single round trip powers the whole admin dashboard.

Byte 27: The shared sync primitive — idempotent upsert

File: SyncService.java

Builds on: Bytes 12, 20, 23 (re-reads PG, writes ES, builds doc from PG)

In plain terms:

This is the heart of synchronization: given an order id, re-read the latest PostgreSQL state, build the search document from it, and overwrite the Elasticsearch copy. Doing it twice changes nothing — that idempotency lets every sync path overlap safely.

The code:

```

/** Shared idempotent upsert used by BOTH strategies + reindex. */
public Map<String, Object> upsertOrderDocument(long orderId) {
    Map<String, Object> doc = buildDocumentTx(orderId); // fresh PG read
    if (doc == null) return Map.of("order_id", orderId, "synced", false, "reason", "not_found");
    es.ensureIndex();
    es.indexDocument(orderId, doc); // doc id = PG order id
    return Map.of("order_id", orderId, "synced", true);
}

public List<Long> bulkUpsertOrders(List<Long> orderIds) { /* same, but batched via _bulk */ }

```

What's happening:

`buildDocumentTx` opens a short read transaction, loads the order with items and customer, and converts it with Byte 23's PG-only builder. The dual-write listener calls the single version; the poller and reindex call the bulk version. All three share this exact code path.

Why it matters:

One shared primitive means the three sync paths cannot drift apart in behavior. Re-reading PG (rather than trusting the message content) is what makes a delayed or redelivered message converge on the truth.

Byte 28: The safety net — polling for changed orders

File: SyncService.java

Builds on: Byte 27 (bulk-syncs whatever this method finds)

In plain terms:

If a RabbitMQ message is ever lost, the worker is down, or Elasticsearch was unreachable, this periodic sweep finds PostgreSQL orders changed recently and pushes them to the index anyway. It is the reason temporary outages never lose syncs.

The code:

```

public Map<String, Object> pollChangedOrders(String strategyTag) {
    try { es.ensureIndex(); }
    catch (Exception ex) {
        return Map.of("checked", 0, "synced", List.of(), "reason", "es_unavailable");
    }
    Instant cutoff = Instant.now().minus(lookbackSeconds, ChronoUnit.SECONDS); // 180s window
    List<Level30Order> rows =
        orders.findByUpdatedAtGreaterThanOrEqualToOrderByDesc(cutoff, PageRequest.of(0, 500));
    List<Long> synced = bulkUpsertOrders(ids); // bulk_failed -> next tick retries
    ...
}

```

What's happening:

The `strategyTag` ("poll") prefixes log lines so you can tell safety-net syncs apart from event-driven `[dual-write]` ones. Every failure mode returns a status map instead of throwing, so the scheduler tick never dies — the next tick simply retries.

Why it matters:

This closes the reliability loop: dual-write is fast but lossy under failure; polling is slower but exhaustive. Together they match Level 2's Celery worker + beat design.

Byte 29: Rebuilding the whole search index from scratch

File: ReindexService.java

Builds on: Bytes 19–20, 27 (index management + bulk upsert)

In plain terms:

Reindex wipes the search index and rebuilds every document from PostgreSQL — the recovery tool after index corruption or a mapping change. Because document ids equal order ids, rebuilding never duplicates anything.

The code:

```
public Map<String, Object> rebuildIndex() {
    es.deleteIndex();
    es.ensureIndex();
    List<Long> ids = orders.findAll().stream().map(Level3Order::getId).sorted().toList();
    List<Long> synced = sync.bulkUpsertOrders(ids);
    es.refresh();
    return Map.of("pg_orders", ids.size(), "es_docs", synced.size(), "index", es.getIndex());
}
```

What's happening:

Delete, recreate with the current mapping, bulk-load every PostgreSQL order, force a refresh so searches see the data immediately, and report `{pg\_orders, es\_docs}` so operators can confirm the counts match. It reuses the same bulk primitive as polling — no separate code path to maintain.

Why it matters:

It is the operational escape hatch: any drift between PostgreSQL and Elasticsearch is fixable with one call, and the count comparison proves the fix worked.

Byte 30: Demo data that builds itself on first boot

File: SeedService.java

Builds on: Bytes 8–13, 29 (writes users/orders/products, then reindexes)

In plain terms:

On first startup the app populates its own demo world — 8 customers, 42 carefully designed orders, 26 products — but only if the databases are empty. Restarts detect existing data and skip, so it never wipes real orders.

The code:

```
public void run(ApplicationArguments args) { // ApplicationRunner: runs once at boot
    if (!enabled) return; // RUN_SEED_ON_START=true in docker
    if (alreadySeeded()) { log.info("Seed data already present; skipping."); return; }
```

```

    seedAll();
}

public boolean alreadySeeded() {
    // 8 users && >=40 orders && >=24 products -> true
    // PG or Mongo unreachable -> true (touch nothing, fail safe)
}

```

What's happening:

`ApplicationRunner` is Spring's "run this once after startup" hook. The guard is deliberately conservative: exact user count plus minimum thresholds, and any infrastructure hiccup means "assume seeded, change nothing" rather than risking a wipe.

Why it matters:

One `docker compose up` gives a fully demoable system with zero manual steps — while making accidental data destruction on restart essentially impossible.

Byte 31: The fixed demo world and its self-checks

File: SeedService.java

Builds on: Byte 30 (the data this flow installs)

In plain terms:

The seed isn't random: the 42 orders are a fixed plan (14 per status, cheap/expensive/old/recent baskets, every customer with 2+ orders) followed by automated quota assertions — and a final trick where the catalog renames the mouse to prove history is frozen.

The code:

```

spec("Morgan", "PROCESSING", 6, line("Wireless Mouse", 1), line("Mechanical Keyboard", 1));
spec("Sam", "SHIPPED", 61, line("4K Monitor 27in", 1));
// ...42 such lines, then:
check(all.size() == 42, ...); // quotas: counts, buckets, totals
void snapshotTestRenameMouse() { // catalog ONLY:
    mongo.updateFirst(query(where("title").is("Wireless Mouse")),
        new Update().set("title", "Wireless Mouse Pro").set("price", 34.99), "products");
}

```

What's happening:

Each spec line is (customer, status, age-in-days, basket). Line items copy the *current* catalog title/price into the order snapshot (Byte 10). The quota checks verify the plan actually landed (status balance, price buckets, per-user minimums, totals reconcile). The rename updates one MongoDB document after everything is indexed — PostgreSQL rows and search docs keep saying "Wireless Mouse".

Why it matters:

Fixed data makes demos and tests reproducible, and the rename is a built-in proof of the snapshot guarantee that the Catalog Admin screen (Byte 55) shows off.

Byte 32: Announcing commits to the sync pipeline

File: OrderSyncPublisher.java

Builds on: Byte 4 (sends to the exchange/queue defined there)

In plain terms:

After an order is safely committed to PostgreSQL, this publisher drops its id into RabbitMQ so the search index can catch up. If the broker is down, it logs and returns false instead of breaking the customer's checkout.

The code:

```
public boolean publishOrderSync(long orderId) {
    try {
        rabbit.convertAndSend(exchange, routingKey, orderId);
        return true;
    } catch (Exception ex) {
        log.error("Failed to publish sync task for order {}", orderId, ex);
        return false;    // PG commit stands; the poller (Byte 28) backfills
    }
}
```

What's happening:

Only the bare order id travels in the message — the consumer re-reads full state from PostgreSQL itself (Byte 27), so messages stay tiny and never go stale. Returning boolean (not throwing) encodes the design rule: sync is best-effort, the database commit is sacred.

Why it matters:

Controllers call this *after* the transaction commits (Byte 40). The false-path is a designed degradation, not an error: checkout succeeds during a broker outage and polling heals the index later.

Byte 33: The event-driven sync worker

File: OrderSyncListener.java

Builds on: Bytes 27, 32 (consumes what the publisher sends, via the upsert primitive)

In plain terms:

This is Strategy A in action: whenever an order id lands in the queue, wake up, rebuild that order's search document from the latest PostgreSQL state, and log the outcome with a `[dual-write]` tag.

The code:

```
@RabbitListener(queues = "${app.sync.queue:level3_order_sync}",
    containerFactory = "syncListenerFactory")
public void onOrderSync(Object message) {
    Long orderId = toOrderId(unwrap(message));
    if (orderId == null) { log.warn("[dual-write] Ignoring unparseable sync message"); return; }
    Map<String, Object> result = sync.upsertOrderDocument(orderId);
    if (Boolean.TRUE.equals(result.get("synced")))
```

```
log.info("[dual-write] Synchronized order {} to Elasticsearch", orderId);
}
```

What's happening:

`@RabbitListener` turns the method into a queue consumer using the retry-enabled factory from Byte 5. Poison (unparseable) messages are logged and skipped rather than retried forever. Genuine failures throw, triggering the Byte 5 retry cycle.

Why it matters:

Live verification showed `[dual-write] Synchronized order 44` within seconds of checkout — this is the fast path that makes "sync'd to search automatically" true for shoppers.

Byte 34: Accepting messages in whatever shape they arrive

File: OrderSyncListener.java

Builds on: Byte 33 (prepares the id that method syncs)

In plain terms:

Messaging frameworks sometimes hand you a converted number, sometimes a raw broker message with JSON bytes. These two helpers normalize every plausible shape — raw bytes, numbers, numeric strings, or id-carrying maps — into a single order id, or null for garbage.

The code:

```
public static Object unwrap(Object message) {
    if (message instanceof Message m) {
        String text = new String(m.getBody(), StandardCharsets.UTF_8).strip();
        try { return Long.parseLong(text); } catch (NumberFormatException ignored) {}
        try { return MAPPER.readValue(text, Map.class); } catch (Exception ignored) {}
        return text;
    }
    return message; // already converted (Long, String, Map) passes through
}

public static Long toOrderId(Object message) {
    if (message instanceof Number n) return n.longValue();
    ...
    if (message instanceof Map<?, ?> m) { /* read "orderId" or "order_id" */ }
    return null;
}
```

What's happening:

`unwrap` peels a raw Spring AMQP `Message` down to its body text, then tries number-first, JSON-map-second, plain-string-last. `toOrderId` handles the converted forms. Both are pure static functions, which is why they are directly unit-testable (Byte 68).

Why it matters:

This defensiveness fixed a real production bug: the listener initially received raw messages, logged "unparseable", and only the poller saved the day. Now both delivery styles converge on the same id.

Byte 35: The timer that drives the safety net

File: PollingSyncScheduler.java

Builds on: Byte 28 (the sweep this timer triggers)

In plain terms:

A three-line class whose only job is to run the polling sync every 15 seconds, starting 20 seconds after boot so the seed and index have time to settle first.

The code:

```
@Scheduled(fixedRateString = "${app.poll.interval-ms:15000}", initialDelayString = "20000")
public void pollChangedOrders() {
    sync.pollChangedOrders("poll");
}
```

What's happening:

`@Scheduled` with `fixedRate` means "every N milliseconds regardless of how long the last run took." Values come from configuration (Byte 2), so operators can tune without rebuilding. This is the Java equivalent of Level 2's Celery Beat entry.

Why it matters:

Strategy B only works if something actually ticks. The initial delay avoids a stampede of sync work while the application is still starting and seeding.

Byte 36: The simplest endpoint — health

File: HealthController.java

Builds on: Byte 1 (a REST controller in the booted web layer)

In plain terms:

`GET /api/health` answers `{ "status": "ok" }`. Docker calls it repeatedly to decide when the API is ready for traffic and when nginx may start sending it requests.

The code:

```
@GetMapping("/api/health")
public Map<String, String> health() {
    return Map.of("status", "ok");
}
```

What's happening:

`@GetMapping` binds the method to GET requests at that path; Spring serializes the returned map to JSON automatically. It touches no database, so it stays green even during outages.

Why it matters:

Nearly every Docker health check and smoke test in the project starts here. If this fails, the problem is the API process itself, not any downstream store.

Byte 37: Listing the demo customers

File: UserController.java

Builds on: Byte 14 (exposes the user repository over HTTP)

In plain terms:

`GET /api/users` returns all seeded customers sorted by id. The frontend's "Log In As" dropdown is populated from this endpoint — there is no real authentication.

The code:

```
@GetMapping
public List<UserOut> listUsers() {
    return users.findAll().stream()
        .sorted((a, b) -> Long.compare(a.getId(), b.getId()))
        .map(u -> new UserOut(u.getId(), u.getName(), u.getEmail()))
        .toList();
}
```

What's happening:

The controller is a thin translation layer: load entities, map each to the `UserOut` DTO shape (Byte 16), return the list. No business logic lives here — that separation is why services are unit-testable without HTTP.

Why it matters:

It demonstrates the intended controller pattern for the whole codebase: validate input, delegate to a service, serialize the result — nothing more.

Byte 38: The catalog API — MongoDB behind four endpoints

File: ProductController.java

Builds on: Byte 24 (delegates every call to ProductService)

In plain terms:

Four endpoints expose the catalog: list (with an `all` flag switching storefront vs admin view), fetch one, create (201), and update. Duplicate SKUs yield 409 and unknown ids 404 via the exception scheme from Bytes 17–18.

The code:

```
@GetMapping
public List<ProductOut> listProducts(
    @RequestParam(name = "all", defaultValue = "false") boolean all) {
    return products.listProducts(all);
}
```

```
@PostMapping
@ResponseStatus(HttpStatus.CREATED)
public ProductOut createProduct(@Valid @RequestBody ProductIn payload) {
    return products.createProduct(payload);
}
```

What's happening:

`@Valid` triggers Byte 16's annotation checks before the method body runs. `@ResponseStatus(CREATED)` returns 201 for successful creation, matching Level 2. The update endpoint's comment ("historical order_items and ES docs are untouched") documents the write-path discipline at the exact place someone might violate it.

Why it matters:

The `all` flag is the only behavioral fork between the shopper view and the catalog-manager view — one endpoint, two audiences, enforced server-side so hidden products truly stay hidden.

Byte 39: Taking orders — the write path with publish-after-commit

File: OrderController.java

Builds on: Bytes 22, 32 (service transaction + publisher)

In plain terms:

Order creation and status changes commit to PostgreSQL first and only **then** announce the sync event. The controller enforces that ordering by publishing after the transactional service call returns.

The code:

```
@PostMapping
@ResponseStatus(HttpStatus.CREATED)
public OrderOut createOrder(@Valid @RequestBody OrderCreate payload) {
    Level3Order created = orders.createOrder(payload); // transaction commits on return
    if (!publisher.publishOrderSync(created.getId())) { // ONLY after commit
        log.warn("Order {} committed but sync publish failed; poller will backfill", created.getId());
    }
    return OrderService.toOrderOut(orders.getOrderOrThrow(created.getId()));
}
```

What's happening:

Because `createOrder` is `@Transactional`, the database commit happens when it returns — so the very next line is provably post-commit. A failed publish degrades to a warning; the order response still goes out with 201 because the customer's purchase is safe regardless of search-index lag.

Why it matters:

This three-line ordering **is** the dual-write guarantee. Reversing it (publish before commit) could index orders that never existed.

Byte 40: Reading orders — PostgreSQL is always the truth

File: OrderController.java

Builds on: Byte 23 (serialization) and Byte 20 (ES fetch for comparison only)

In plain terms:

Order listing, order details, and the sync-status indicator all read canonical data from PostgreSQL. Elasticsearch is consulted only to **compare** timestamps for the "In sync / Pending" badge — never as a data source.

The code:

```
@GetMapping
public List<OrderOut> listOrders(
    @RequestParam(name = "user_id", required = false) Long userId,
    @RequestParam(name = "status", required = false) String status,
    @RequestParam(name = "page", defaultValue = "1") int page,
    @RequestParam(name = "size", defaultValue = "20") int size) {
    // page/size bounds + status normalization -> 422 on violation
    return orders.listOrders(userId, filter, page, size).stream()
        .map(OrderService::toOrderOut).toList();
}

@GetMapping("/{orderId}/sync")
public OrderSyncStatus orderSyncStatus(@PathVariable long orderId) {
    // pg_updated_at from PG, es_updated_at from ES doc (null if missing),
    // in_sync = equal non-null timestamps
}
```

What's happening:

`user\_id` filtering powers "My Orders"; `status` filtering shares Byte 21's validation (bad values → 422, same as Level 2). Pagination is newest-first with explicit bounds. The sync endpoint compares ISO timestamps as strings — simple, exact, and honest about "missing".

Why it matters:

"Reads come from PostgreSQL, never Elasticsearch" is the rule that keeps the UI correct during search outages. The sync badge turns an invisible background process into something users and admins can see.

Byte 41: Admin search endpoint — Elasticsearch only

File: SearchController.java

Builds on: Bytes 21, 26 (status validation + search execution)

In plain terms:

`POST /api/search/orders` accepts the admin screen's filters, validates them (bad status/sort/page/size → 422), runs the search, and converts an Elasticsearch outage into a 503 — while PostgreSQL orders stay perfectly safe.

The code:

```

@PostMapping("/orders")
public Map<String, Object> searchOrdersPost(@RequestBody(required = false) OrderSearchRequest payload)
{
    List<String> statuses = OrderStatuses.requireValidList(req.statuses());
    ...
    try {
        return search.searchOrders(req.q(), statuses, ...);
    } catch (IllegalStateException ex) {
        throw new ResponseStatusException(HttpStatus.SERVICE_UNAVAILABLE,
            "Search unavailable: " + ex.getMessage()); // ES down -> 503
    }
}

```

What's happening:

A null body is treated as "no filters" rather than an error. Every numeric bound is checked manually because the DTO uses nullable wrapper types. The 503 mapping is the user-visible half of the split-brain story: search can be down while checkout keeps working.

Why it matters:

The frontend shows a specific "search index unavailable, orders are safe" message for 503 (Byte 53) — that precise UX is only possible because this endpoint distinguishes "bad request" (422) from "search down" (503).

Byte 42: The reindex button as an API

File: AdminController.java

Builds on: Byte 29 (exposes the rebuild over HTTP)

In plain terms:

`POST /api/admin/reindex` triggers a full index rebuild and reports `{pg\_orders, es\_docs, index}`. It is the documented, one-command recovery tool — no scripts, no container shells needed.

The code:

```

@PostMapping("/reindex")
public Map<String, Object> reindex() {
    return reindex.rebuildIndex();
}

```

What's happening:

The entire controller is one delegation call. All logic (delete, recreate, bulk-load, refresh, count) lives in the tested service, so the endpoint adds no new behavior to verify.

Why it matters:

Operations tooling belongs behind the API, not in runbooks: the rebuild is authenticated, logged, and callable the same way in every environment. The returned counts let the caller prove the rebuild worked.

Byte 43: The frontend's phone book for the API

File: client.js

Builds on: Bytes 36–42 (wraps every endpoint above)

In plain terms:

This small JavaScript module is the only place the UI touches the network. Each backend area gets a named object (`usersApi`, `productsApi`, `ordersApi`, `searchApi`) with one function per endpoint, all going through a shared axios instance.

The code:

```
const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL || '', // same-origin /api in Docker
  timeout: 15000
})

export const ordersApi = {
  create: (payload) => api.post('/api/orders', payload).then(r => r.data),
  list: (params = {}) => api.get('/api/orders', { params }).then(r => r.data),
  get: (id) => api.get(`/api/orders/${id}`).then(r => r.data),
  updateStatus: (id, status) => api.patch(`/api/orders/${id}/status`, { status }).then(r => r.data),
  syncStatus: (id) => api.get(`/api/orders/${id}/sync`).then(r => r.data)
}
```

What's happening:

`baseURL` is empty by default, so calls go to the same origin — inside Docker that means nginx, which proxies `/api` to the Java backend (Byte 61). Every helper unwraps `r.data`, so views never see HTTP plumbing. The comment about the storefront passing `{ all: true }` for admin documents the Byte 38 fork.

Why it matters:

Centralizing API access means the backend move from Python to Java required zero view changes — only this wiring (and the dev proxy port) identifies the backend at all.

Byte 44: Cart and identity live in the browser, not the database

File: store.js

Builds on: Byte 43 (uses the API only for user identity, never for cart)

In plain terms:

There is no cart table anywhere. The shopping cart, the chosen customer, and recently placed order ids live in the browser's localStorage, managed by this tiny reactive store module.

The code:

```
export const store = reactive({
  user: savedUser,
  setUser(u) { this.user = u; localStorage.setItem('l2-user', JSON.stringify(u)) },
  cart: reactive(saved),
  clearCart() { this.cart.splice(0, this.cart.length); persist() }
})
```

```
export function addToCart(product, qty = 1) {
  const line = store.cart.find(l => l.id === product.id)
  if (line) line.qty += qty
  else store.cart.push({ id: product.id, title: product.title, price: product.price, qty })
}
```

What's happening:

`reactive` makes the cart update the UI automatically when it changes. Every mutation persists to localStorage so refreshes keep the cart. `recordOrder`/`recentOrderIds` remember placed order ids so "My Orders" can re-fetch them from PostgreSQL later (Byte 52).

Why it matters:

This is a deliberate architecture decision, not a shortcut: no cart service means no cart synchronization problem. The server only ever sees a finished checkout — a list of product ids and quantities.

Byte 45: The map of screens

File: index.js

Builds on: Bytes 49–55 (the views this file connects)

In plain terms:

The router maps seven URLs to seven screens: storefront, product page, checkout, my orders, admin search, admin order detail, and catalog manager. No guards, no nesting — just a flat table.

The code:

```
export default createRouter({
  history: createWebHistory(),
  routes: [
    { path: '/', component: Storefront },
    { path: '/products/:id', component: ProductDetail },
    { path: '/checkout', component: Checkout },
    { path: '/orders', component: MyOrders },
    { path: '/admin', component: AdminSearch },
    { path: '/admin/orders/:id', component: AdminOrderDetail },
    { path: '/catalog', component: CatalogAdmin }
  ]
})
```

What's happening:

`createWebHistory` gives clean URLs without `#` fragments, which is why nginx needs the `try\_files ... /index.html` fallback (Byte 61) — deep links must serve the app and let it route client-side.

Why it matters:

The route table is the product's information architecture in one glance. Each route's data source (Mongo catalog vs PostgreSQL orders vs Elasticsearch search) is labeled on the screen itself, teaching the architecture while you click.

Byte 46: Three lines that start the whole UI

File: main.js

Builds on: Bytes 45, 48 (router and root component it wires together)

In plain terms:

The entire frontend boots in three statements: create the Vue app from the root component, plug in the router, and attach it to the page's `#app` element — plus one global stylesheet import.

The code:

```
import { createApp } from 'vue'
import App from './App.vue'
import router from './router/index.js'
import './styles.css'

createApp(App).use(router).mount('#app')
```

What's happening:

`createApp(App)` builds the application instance, `.use(router)` installs navigation, `.mount('#app')` renders into the `

Why it matters:

If the page is blank, this file plus index.html is the first place to look: a wrong mount id or a failed router import fails silently here and nothing else ever runs.

Byte 47: The shell around every screen

File: App.vue

Builds on: Bytes 37, 43–44 (loads users, holds the shared store)

In plain terms:

The root component renders the promo strip, header (logo, search box, customer picker, nav, cart pill), the current screen, and the footer. It also loads the customer list once and auto-selects the first customer so every screen has an identity to work with.

The code:

```
<select class="input" v-model="userId" @change="persistUser" title="Log In As">
  <option :value="null">Log In As...</option>
  <option v-for="u in users" :key="u.id" :value="u.id">{{ u.name }}</option>
</select>
<router-link class="cart-pill" to="/checkout">Cart<span class="count">{{ cartCount }}</span></router-link>
```

What's happening:

The customer picker writes into the shared store (Byte 44), so choosing "Wendy Wireless" here affects checkout and order history everywhere. The header search jumps to the storefront with a `?q=` parameter the store watches. The footer states the architecture ("MongoDB · PostgreSQL · Elasticsearch") on every page.

Why it matters:

"Login" here is just selecting a seeded identity — understanding that prevents confusion about missing auth. The catch-to-empty-list pattern also means the shell renders even when the API is down.

Byte 48: The shop window — client-side catalog browsing

File: Storefront.vue

Builds on: Bytes 38, 43–44 (lists products, adds to cart)

In plain terms:

The home page loads all active products once, then filters, sorts, and deals-badges them entirely in the browser: category tiles with live counts, an "Under \$25" rail, text search, and price/name sorting.

The code:

```
const filtered = computed(() => {
  const needle = q.value.trim().toLowerCase()
  let list = products.value.filter(p =>
    (!category.value || p.category === category.value) &&
    (!onlyDeals.value || Number(p.price) < 25) &&
    (!needle || [p.title, p.description, p.skus, (p.tags || []).join(' ')].join(' ').includes(needle)))
  if (sort.value === 'price_asc') list = [...list].sort((a, b) => a.price - b.price)
  ...
})
```

What's happening:

`computed` values recalculate automatically when their inputs change, so typing in the search box re-filters with no extra code. The storefront calls `productsApi.list({})` — no `all` flag — so discontinued products never reach this screen. It never writes anything: "Add to Cart" only touches the browser store.

Why it matters:

This screen demonstrates the read-only catalog path: MongoDB → API → UI, with zero order or search involvement. Its `?q=/?category=/?deals=` URL syncing also makes filtered views shareable links.

Byte 49: One product, full detail, add to cart

File: ProductDetail.vue

Builds on: Bytes 38, 44 (single-product fetch + cart)

In plain terms:

The product page fetches one catalog item by id, shows its specs table (attributes and variants included), and offers quantity selection with add-to-cart — or a straight jump to checkout.

The code:

```
onMounted(async () => {
  try {
    product.value = await productsApi.get(route.params.id)
  } catch (e) {
    error.value = 'Product not found.'
  }
})
```

What's happening:

The id comes from the URL (`/products/:id` in Byte 45). A 404 from the API becomes a friendly message instead of a broken page. The spec table renders the free-form MongoDB `attributes` map generically, so new product types display correctly with no code changes.

Why it matters:

It proves the catalog's schemaless payoff end to end: a product with never-before-seen attributes still renders, because neither the API DTO (a plain map) nor this table assumes a fixed schema.

Byte 50: Turning a cart into a PostgreSQL order

File: Checkout.vue

Builds on: Bytes 39, 43–44 (creates the order, clears the cart, records the id)

In plain terms:

Checkout shows cart lines with quantity steppers, picks a customer, and places the order with one POST: `{user\_id, items: [{product\_id, quantity}]}`. Success clears the cart; backend error text (`detail`) is shown verbatim.

The code:

```
const order = await ordersApi.create({
  user_id: Number(userId.value),
  items: store.cart.map(l => ({ product_id: l.id, quantity: l.qty }))
})
msg.value = `Order #${order.id} placed (${Number(order.total_amount).toFixed(2)}). Syncing to search...`
recordOrder(order.id)
store.clearCart()
```

What's happening:

Note what is *not* sent: no prices, no titles — the server re-validates against live MongoDB and snapshots its own copies (Byte 22), so a tampered browser price is ignored. `recordOrder` saves the id locally for the My Orders screen. The "Syncing to search..." message sets the expectation that admin search updates seconds later.

Why it matters:

This is the only screen that writes transactional data, and it writes only to PostgreSQL via the API. The badge in its header ("MongoDB validation → PostgreSQL order") names the exact cross-store flow.

Byte 51: Your orders, read canonically from PostgreSQL

File: MyOrders.vue

Builds on: Bytes 40, 44 (order reads + local order ids)

In plain terms:

My Orders prefers the logged-in customer's full PostgreSQL history, then tops up with any extra orders this browser placed (e.g. for other customers), deduplicating by id. Every card links to the admin detail view.

The code:

```
if (store.user) {
  for (const o of await ordersApi.list({ user_id: store.user.id, size: 50 })) {
    if (!seen.has(o.id)) { seen.add(o.id); out.push(o) }
  }
}
for (const id of recentOrderIds()) {
  if (seen.has(id)) continue
  try { out.push(await ordersApi.get(id)); seen.add(id) } catch (e) { /* skip */ }
}
```

What's happening:

Two sources merge into one `Set`-deduplicated list: server-side history first, browser-local ids second (each fetched individually). Failures in either source degrade gracefully to whatever loaded — the screen never blanks because one path failed.

Why it matters:

It exercises the `GET /api/orders?user\_id=` filter and per-order GET exactly as designed: canonical PostgreSQL reads, with the browser ids only as a discovery mechanism, never as data.

Byte 52: Admin search over the Elasticsearch projection

File: AdminSearch.vue

Builds on: Bytes 26, 41 (search envelope + endpoint contract)

In plain terms:

The admin screen posts text/status/date/total/sort/page filters to `/api/search/orders` and renders the results table plus KPI cards (filtered revenue, per-status counts, total hits) straight from the response's aggregations.

The code:

```
const out = await searchApi.orders({
  q: f.q || null,
  statuses: f.statuses.length ? f.statuses : null,
  date_from: f.date_from || null,
  date_to: f.date_to ? f.date_to + 'T23:59:59' : null,
  ...
})
```

```
results.value = out.results; aggs.value = out.aggs || {}; total.value = out.total
```

What's happening:

Empty filters are sent as `null` (meaning "ignore this filter"), and the end date is extended to end-of-day so "to Oct 5" includes Oct 5th orders. The 503 branch is the split-brain UX: it explicitly reassures that orders are safe when only search is down.

Why it matters:

This screen is the *only* consumer of Elasticsearch. If it is wrong or slow, the bug is in the projection or sync — customer-facing flows are provably unaffected, which is the entire point of the architecture.

Byte 53: Order truth with a sync badge and status control

File: AdminOrderDetail.vue

Builds on: Bytes 39–40 (status update + sync comparison endpoints)

In plain terms:

The detail page shows the canonical PostgreSQL receipt (snapshot lines with frozen prices), a live "In sync ✓ / Pending..." badge comparing PG vs ES timestamps, and a fulfillment control that PATCHes the status.

The code:

```
order.value = await ordersApi.get(id) // PG truth
sync.value = await ordersApi.syncStatus(id).catch(() => null) // PG vs ES badge
async function updateStatus() {
  order.value = await ordersApi.updateStatus(id, nextStatus.value) // PG commit + sync publish
  msg.value = 'Status saved to PostgreSQL and sync queued → search index updates shortly.'
  sync.value = await ordersApi.syncStatus(id).catch(() => null)
}
```

What's happening:

Details come from `GET /api/orders/{id}` (never the search index), while the badge comes from the sync endpoint (Byte 40). After a status change the badge is re-fetched immediately — it typically reads "Pending..." for a few seconds until the dual-write lands, making the async pipeline visible.

Why it matters:

This one screen demonstrates the full write discipline: PostgreSQL owns the data, the status control triggers the sync, and the badge proves convergence — the architecture made clickable.

Byte 54: Managing the MongoDB catalog safely

File: CatalogAdmin.vue

Builds on: Bytes 38, 49 (product update/create + variant preservation)

In plain terms:

The catalog manager lists **all** products including discontinued ones, and its create/edit form writes only catalog fields back to MongoDB — explicitly promising that historical orders are untouched, with the Mouse/Pro rename as living proof.

The code:

```
products.value = await productsApi.list({ all: true }).catch(() => [])
const payload = {
  sku: form.sku, title: form.title, ..., price: Number(form.price),
  tags: form.tags.split(',').map(t => t.trim()).filter(Boolean),
  attributes: { brand: form.brand },
  variants: editing.value?.id
    ? (products.value.find(p => p.id === editing.value.id)?.variants || [])
    : [{ name: 'Standard', sku: form.sku, price_adjustment: 0 }],
  active: form.active
}
```

What's happening:

`{ all: true }` is the admin fork from Byte 38. Editing preserves existing variants (the form doesn't edit them); creation seeds one Standard variant so the DTO is always complete. Tags are converted between comma-string (form) and array (API) at the boundary. Backend errors surface via the shared `detail` envelope.

Why it matters:

The header text states the snapshot guarantee in plain language next to the renamed product — the UI itself teaches why catalog edits are safe, closing the loop opened by the seed's rename trick (Byte 31).

Byte 55: Keyword-driven product illustrations with zero network cost

File: productArt.js

Builds on: Bytes 48–49, 51, 54 (every screen that shows a product image)

In plain terms:

There are no product image files. This module matches a product's title/category/tags against keyword patterns (mouse, keyboard, monitor...) and returns a purpose-built inline SVG picture as a data URI, with a generic chip illustration as fallback.

The code:

```
const TYPES = [
  { key: 'mouse', test: /mouse/i, bg: ['#1e3a8a', '#3b82f6'], glyph: 'mouse' },
  { key: 'keyboard', test: /keyboard/i, bg: ['#4c1d95', '#8b5cf6'], glyph: 'keyboard' },
  ...
]

function typeFor(product = {}) {
  const hay = [product.title, product.category, ...(product.tags || [])].filter(Boolean).join(' ')
  return TYPES.find(t => t.test.test(hay)) || { key: 'chip', bg: [...], glyph: 'chip' }
}
```

```
export function productArt(product) { /* gradient bg + glyph -> data:image/svg+xml URI */ }
export function artByTitle(title = '') { return productArt({ title }) }
export const CATEGORY_ART = { peripherals: 'keyboard', /* ... */ }
```

What's happening:

`typeFor` concatenates the searchable text fields and returns the first matching type; `productArt` wraps that type's glyph SVG in a gradient card and URL-encodes it into an ``-ready URI. `artByTitle` covers order lines where only the snapshot title is known (renamed products still get sensible art).

Why it matters:

Product imagery can never 404, needs no hosting, and automatically handles new products — including the renamed "Wireless Mouse Pro", which still matches `/mouse/i`. It is purely presentational: zero backend involvement.

Byte 56: The look of the whole shop in one file

File: styles.css

Builds on: Bytes 47–54 (classes every view references)

In plain terms:

One global stylesheet defines the entire visual language — topbar, hero, product grid, cards, toolbar, status pills, tables, KPI tiles, and skeleton loaders — so all seven screens look like one product with no per-view CSS.

The code:

```
.pcard { /* product/order card container */ }
.status.PENDING { /* amber pill */ }
.status.PROCESSING { /* blue pill */ }
.status.SHIPPED { /* green pill */ }
.source-badge.pg / .mongo / .es { /* per-screen data-source labels */ }
.skel-grid / .skel { /* loading shimmer placeholders */ }
```

What's happening:

Status pills are color-coded by class name bound directly to the order status string (`:class="o.status"`), which is why the uppercase canonical statuses matter visually too. The `source-badge` variants label each screen's data store (PG green, Mongo purple, ES orange). Skeleton grids hold layout while data loads.

Why it matters:

Level 3 kept this file byte-identical on purpose: the backend rewrite is proven invisible when the UI — down to the pixels — doesn't change.

Byte 57: The three small files that wire the frontend

File: index.html

Builds on: Byte 46 (provides the `#app` mount point and loads main.js)

In plain terms:

Three tiny configs make the frontend buildable and routable: the HTML shell with its mount point, the Vite dev setup pointing `/api` at the Java backend's port, and the package manifest renaming the app to Level 3.

The code:

```
<title>Level 3 E-Commerce – Orders & Search (Java API)</title>
<div id="app"></div>
<script type="module" src="/src/main.js"></script>
```

What's happening:

`index.html` is the only HTML file: mount div plus module script (Vite rewrites it at build). The Vite proxy forwards local `/api` calls to the Java API on 8003 during development — in Docker this file is irrelevant because nginx proxies instead (same-origin, Byte 61). The package rename marks the port from Level 2.

Why it matters:

These are the *only* frontend files that changed for Level 3 (plus the title). That diff is the proof that API compatibility held: new backend, same UI, three lines of wiring.

Byte 58: Isolated infrastructure in one command

File: docker-compose.yml

Builds on: Byte 2 (every env value here flows into application.properties)

In plain terms:

The infrastructure half of the compose file starts four isolated data services — PostgreSQL, MongoDB, RabbitMQ, Elasticsearch — each with a Level 3 container name, host port, volume, and health check, all on a private network.

The code:

```
level3-postgres:
  image: postgres:16-alpine
  container_name: level3-postgres
  ports: ["5445:5432"]
  volumes: [level3_pgdata:/var/lib/postgresql/data]
level3-mongodb:
  ports: ["27018:27017"]      # Level 2 uses 27017 – no clash
level3-rabbitmq:
  ports: ["5683:5672", "15773:15672"]
level3-elasticsearch:
  ports: ["9211:9200"]
networks: [level3-net]      # private; Level 2 has its own network
```

What's happening:

Host ports are all shifted away from Level 2's (5435→5445, 5673→5683, 9201→9211, 27017→27018) so both stacks run simultaneously. Named volumes (`level3\_`) persist data separately from Level 2's volumes. Health checks gate startup order: nothing proceeds until each store answers.

Why it matters:

This file is the physical isolation guarantee. Same images as Level 2, but nothing — names, ports, volumes, network — is shared, so Level 3 can never read or overwrite Level 2 data.

Byte 59: API and frontend serving on the private network

File: docker-compose.yml

Builds on: Byte 58 (starts after the healthy stores above)

In plain terms:

The second half of the compose file builds and starts the Java API (with seed enabled) and the nginx frontend, wiring the API to the Level 3 stores by container name and exposing only two host ports.

The code:

```
level3-java-api:
  build: { context: ./backend }
  environment:
    POSTGRES_URL: jdbc:postgresql://level3-postgres:5432/ecom_level3
    MONGO_URL: mongodb://level3-mongodb:27017/ecom_level3
    RABBITMQ_HOST: level3-rabbitmq
    ELASTICSEARCH_URL: http://level3-elasticsearch:9200
    ELASTICSEARCH_INDEX: level3_orders
    SYNC_QUEUE: level3_order_sync
    RUN_SEED_ON_START: "true"
  ports: ["8003:8080"]
  depends_on: { level3-postgres: { condition: service_healthy }, ... }

level3-nginx:
  build: { context: ., dockerfile: nginx/Dockerfile }
  ports: ["8082:80"]
  depends_on: { level3-java-api: { condition: service_healthy } }
```

What's happening:

Inside the compose network, services reach each other by container name on internal ports (`level3-postgres:5432`), while the host sees only 8003 (API) and 8082 (shop). `depends\_on` with `service\_healthy` plus the entrypoint wait (Byte 60) means the API boots into ready infrastructure, then seeds, then nginx starts serving.

Why it matters:

`docker compose up -d --build` is the whole deployment story: build both images, start in dependency order, seed once, serve. The `down -v` variant resets Level 3 alone because every resource here is project-scoped.

Byte 60: Building the Java container and waiting for its world

File: Dockerfile (backend)

Builds on: Byte 3 (compiles the Maven project inside Docker)

In plain terms:

A two-stage build compiles the jar with Maven, then ships only the jar plus a small Java runtime into the final image. The entrypoint script waits for all four infrastructure ports before launching the app.

The code:

```
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /build
COPY pom.xml ./
RUN mvn -q dependency:go-offline -DskipTests || true # cache dependencies
COPY src ./src
RUN mvn -q -DskipTests package

FROM eclipse-temurin:17-jre
RUN apt-get update && apt-get install -y --no-install-recommends curl ...
COPY --from=build /build/target/level3-ecommerce-3.0.0.jar app.jar
ENTRYPOINT ["/docker-entrypoint.sh"]
CMD ["java", "-jar", "app.jar"]
```

What's happening:

Stage one caches dependencies separately from source so rebuilds are fast when only code changes. The slim runtime image carries no JDK or Maven. The entrypoint's TCP loop covers the gap between "container exists" and "service answers" that compose health checks alone can miss on first boot.

Why it matters:

Without the wait, the API would crash-loop against not-yet-ready databases. With multi-stage, the shipped image stays small and contains no build tools.

Byte 61: Serving the shop and routing to the API

File: nginx.conf

Builds on: Bytes 45–46 (needs the SPA fallback) and Byte 59 (proxies to the API)

In plain terms:

Nginx serves the built Vue files and forwards `/api/*` plus the Swagger endpoints to the Java container; anything else falls back to `index.html` so client-side routes work on refresh and deep links.

The code:

```
upstream javaapi { server level3-java-api:8080; }

location /api/ {
    proxy_pass http://javaapi;
    proxy_set_header Host $host;
    ...
}
location /v3/api-docs { proxy_pass http://javaapi; }
location /swagger-ui.html { proxy_pass http://javaapi; }
location /swagger-ui/ { proxy_pass http://javaapi; }
location / { try_files $uri $uri/ /index.html; }
```

What's happening:

The nginx build (multi-stage like Byte 60, but Node-based) compiles the SPA with an empty `VITE\_API\_URL`, baking in same-origin `/api` calls — which land here and are proxied to Java. The `/docs` and `/openapi.json` paths from Level 2's Python setup are replaced by Springdoc's `/v3/api-docs` + Swagger UI paths.

Why it matters:

Same-origin serving means the browser never needs CORS in production and the API needs no public port for the shop to work. The `try\_files` fallback is what makes bookmarked `/admin/orders/44` URLs load instead of 404ing.

Byte 62: Example settings with no secrets

File: .env.example

Builds on: Bytes 2, 58–59 (documents every knob those files consume)

In plain terms:

A commented template listing every Level 3 environment value — database URLs, queue, index, ports, seed flag — with safe example values only, so operators know what to set without ever seeing a real password.

The code:

```
POSTGRES_URL=jdbc:postgresql://localhost:5445/ecom_level3
MONGO_URL=mongodb://localhost:27018/ecom_level3
RABBITMQ_PORT=5683
ELASTICSEARCH_URL=http://localhost:9211
ELASTICSEARCH_INDEX=level3_orders
SYNC_QUEUE=level3_order_sync
SYNC_EXCHANGE=level3_orders
RUN_SEED_ON_START=true
API_PORT=8003
FRONTEND_URL=http://localhost:8082
```

What's happening:

It mirrors `application.properties` one-to-one (same variable names, host-port variants for running outside Docker). Nothing here is loaded automatically — it is documentation you copy to `.env` and adjust.

Why it matters:

Every Level 3–specific name from the isolation table (Byte 58) appears here in one place, making accidental reuse of a Level 2 value easy to spot in review.

Byte 63: The project's front page

File: README.md

Builds on: Everything (summarizes the whole system for a newcomer)

In plain terms:

The README is the operator's quick-start: one start command, the URL table, the endpoint table, the data-flow diagram, seed/reindex instructions, test commands, and the Level 2 vs Level 3 isolation table.

The code:

```
docker compose up -d --build
# Storefront: http://localhost:8082 | Java API: http://localhost:8003
# Swagger UI: http://localhost:8003/swagger-ui.html
Invoke-RestMethod -Method Post -Uri "http://localhost:8003/api/admin/reindex"
```

What's happening:

Unlike code comments, this file speaks to humans first: what to run, where to click, how to recover. The isolation table (ports, databases, queues, indexes, volumes) is the checklist an auditor uses to confirm Level 2 is untouched.

Why it matters:

A project nobody can start is a project nobody keeps. This file compresses the entire operational knowledge of Bytes 58–62 into one page.

Byte 64: Proving the order math with a real database

File: OrderServiceTest.java

Builds on: Byte 22 (tests the creation logic with H2 standing in for PostgreSQL)

In plain terms:

These tests run the real `OrderService` against an in-memory H2 database with MongoDB mocked, verifying snapshot totals, rejection of bad carts (missing/inactive/duplicate products, unknown user), history immunity to catalog renames, status rules, and listing filters.

The code:

```
@DataJpaTest
@Import(OrderService.class)
class OrderServiceTest {
    @MockBean
    private ProductService products;    // MongoDB faked with Mockito answers

    @Test
    void orderSnapshotsTitleAndPriceAndTotal() {
        OrderOut out = ...createOrder(new OrderCreate(wendyId(),
            List.of(new OrderItemIn("m1", 2), new OrderItemIn("k1", 1))));
        assertEquals(Math.round((24.99 * 2 + 89.99) * 100.0) / 100.0, out.totalAmount());
        assertEquals("Wireless Mouse", out.items().get(0).title());
    }
}
```

What's happening:

`@DataJpaTest` boots only the JPA slice with H2 — no Docker, no Mongo, no broker — while `@MockBean` replaces the Mongo-backed service with scripted answers. Each test gets a fresh database via the `clean`-style setup, mirroring the Level 2 Python test cases one by one.

Why it matters:

The trickiest business rules (snapshot integrity, atomic rollback leaving zero rows) are proven here in seconds. If someone breaks order math, this suite fails before any container is built.

Byte 65: Proving both sync strategies without any servers

File: SyncServiceTest.java

Builds on: Bytes 27–28 (tests the upsert and poll primitives)

In plain terms:

Three tests pin down synchronization: double-upserting the same order writes one document (idempotency), an Elasticsearch failure surfaces as an exception while the PostgreSQL row survives (no rollback), and the poll sweep picks up changed orders in bulk.

The code:

```
@Test
void dualWriteUpsertIsIdempotent() {
    assertTrue((Boolean) sync.upsertOrderDocument(id).get("synced"));
    assertTrue((Boolean) sync.upsertOrderDocument(id).get("synced"));
    assertEquals(List.of(id), List.copyOf(store.keySet())); // exactly one doc
}

@Test
void esDownSurfacesRetryWithoutRollback() {
    doThrow(new IllegalStateException("Elasticsearch unavailable")).when(es).indexDocument(...);
    assertThrows(IllegalStateException.class, () -> sync.upsertOrderDocument(id));
    assertEquals(1, orders.findById(id).stream().count()); // PG row intact
}
```

What's happening:

Elasticsearch is a Mockito mock capturing into a `store` map, so assertions inspect exactly what would have been indexed. The ES-down test is the executable version of the split-brain promise: sync fails loudly, data stays safe.

Why it matters:

These are the reliability guarantees as runnable proof. The retry-and-poller machinery (Bytes 5, 28, 35) is only trustworthy because these behaviors are locked in by tests.

Byte 66: Pure logic tests with no Spring at all

File: SearchQueryTest.java

Builds on: Bytes 25–26 (query builder and envelope parser)

In plain terms:

Two companion test files check code that needs no database whatsoever: the Elasticsearch query translator (multi_match, terms, ranges, match-all, order-id shortcut) and the order-to-document snapshot builder (titles, prices, customer fields).

The code:

```
@Test
void buildQueryUsesMultiMatchTermsRangeAndAggs() {
    Map<String, Object> q = SearchService.buildEsQuery(
        "Wireless", List.of("PENDING", "SHIPPED"), "2026-01-01", "2026-12-31", 10.0, 200.0);
    assertTrue(q.toString().contains("multi_match"));
    assertTrue(q.toString().contains("terms"));
    assertTrue(q.toString().contains("range"));
}
// OrderDocumentTest builds an in-memory order and asserts the doc keeps
// "Wireless Mouse" @ 24.99 even after the catalog object is renamed.
```

What's happening:

`OrderDocumentTest` constructs entities with `new` (no persistence) and calls the static converters — the fastest possible test. `SearchQueryTest` feeds a fake Elasticsearch client returning canned JSON and asserts the parsed `{total, aggs}` envelope. (Both files share this byte because both test pure functions.)

Why it matters:

Pure-function tests run in milliseconds and pinpoint failures exactly: a broken query shape fails here, not somewhere deep in a Docker stack. They also document the ES query contract by example.

Byte 67: Proving the listener understands every message shape

File: OrderSyncListenerTest.java

Builds on: Byte 34 (tests unwrap/toOrderId directly)

In plain terms:

Four tests feed the listener raw broker bytes (`"43"`), an already-converted number, and garbage — verifying the first two trigger a sync for the right order id while garbage is ignored without touching Elasticsearch.

The code:

```
private static Message raw(String body) {
    MessageProperties props = new MessageProperties();
    props.setContentType("application/json");
    return new Message(body.getBytes(StandardCharsets.UTF_8), props);
}

@Test
void rawLongBodySyncs() {
    new OrderSyncListener(sync).onOrderSync(raw("43"));
    verify(sync).upsertOrderDocument(43L);
}
```

```
@Test
void garbageIsIgnoredWithoutThrowing() {
    new OrderSyncListener(sync).onOrderSync(raw("not-json{{{}}"));
    verifyNoInteractions(sync);
}
```

What's happening:

The `SyncService` is mocked, so the test asserts **routing** (correct id extracted, sync invoked or not) rather than indexing. Building a real `Message` with JSON content-type reproduces the exact production failure from Byte 70's gotcha.

Why it matters:

This is the regression net for the raw-message bug: if anyone simplifies `unwrap`/`toOrderId` back to assuming converted payloads, these tests fail immediately.

Byte 68: Gotcha — calling your own @Transactional method does nothing

File: SeedService.java

Builds on: Bytes 30–31 (the seed flow this nearly broke)

In plain terms:

Spring transactions work through a proxy wrapped around your bean — but a method calling **another method on the same object** bypasses the proxy, so annotations like `@Transactional` are silently ignored and lazy database collections explode.

The code:

```
// BROKEN: verifyQuotas() is @Transactional, but this self-call skips the proxy
// -> LazyInitializationException: could not initialize proxy - no Session
// seedAll() { ... txTemplate.executeWithoutResult(tx -> { ... }); verifyQuotas(); }

// FIXED: verify inside the ambient transaction, after flush + clear
em.flush();
em.clear();
verifyQuotas(); // runs inside txTemplate's transaction -> lazy items load fine
```

What's happening:

The first Docker boot crashed with `LazyInitializationException` because `verifyQuotas` touched `order.getItems()` with no open session. Moving the call inside the `txTemplate` block (plus `flush`/`clear` so checks read fresh rows) fixed it. The container then restarted, saw partial seed data, and skipped — which is why the fix also required a volume reset to get a clean first seed.

Why it matters:

This is the most common Spring transaction pitfall, and it fails at runtime, not compile time. Rule of thumb: never rely on `@Transactional` for self-invocation — use the template, the proxy (self-injection), or restructure.

Byte 69: Gotcha — the listener received raw messages, not converted ones

File: OrderSyncListener.java

Builds on: Bytes 5, 33 (the factory and listener involved)

In plain terms:

The queue consumer was written expecting nicely converted order ids, but in production it received raw broker `Message` objects — so every sync was logged "unparseable" and only the polling safety net kept search updated. The fix parses both shapes.

The code:

```
// What arrived: (Body:'43' MessageProperties [headers={__TypeId__=java.lang.Long},
//   contentType=application/json, ...]) -> toOrderId returned null -> skipped
// Fix: unwrap() reads m.getBody() as UTF-8, tries Long, then JSON map, then String.
```

What's happening:

Whether the container's message converter applies depends on subtle framework wiring; assuming one shape is fragile. The defensive `unwrap` + `toOrderId` pair accepts raw bytes, converted numbers, numeric strings, and id-maps alike, and the Byte 67 tests lock all four shapes in.

Why it matters:

Without the poller masking it, this bug would have meant "checkout works, search never updates." Lesson: when consuming from a broker, always handle the raw `Message` form — and keep an independent test that constructs one.

Byte 70: Gotcha — JPA association paths need an underscore

File: OrderRepository.java

Builds on: Byte 12 (the repository this bit)

In plain terms:

Spring Data derives queries from method names, but `findByUserId` looks for a *field* called `userId` — which doesn't exist (the entity has a `user` association with an `id`). The correct spelling `findByUser\_Id` walks the association, and the wrong one fails at startup with "Could not resolve attribute."

The code:

```
// FAILS: InvalidDataAccessApiUsageException: Could not resolve attribute 'userId'
Page<Level3Order> findByUserId(Long userId, Pageable pageable);

// WORKS: underscore traverses user -> id
Page<Level3Order> findByUser_Id(Long userId, Pageable pageable);
Page<Level3Order> findByUser_IdAndStatus(Long userId, String status, Pageable pageable);
```

What's happening:

The underscore is Spring Data's property-traversal operator: `User` (association) then `Id` (its key). This broke a slice test immediately, which is exactly the cheap feedback `@DataJpaTest` exists for.

Why it matters:

Derived-query names are effectively untested SQL until something runs them. The rule: any repository method you add must be executed by at least one test, or it may be broken stringly-typed SQL waiting for production.

PUTTING IT TOGETHER

A shopper browses MongoDB-backed products (Bytes 11, 48), checks out through a transactional service that freezes prices into PostgreSQL snapshots (Bytes 10, 22, 50), and the controller announces the committed order to RabbitMQ (Bytes 32, 39). The listener turns that event into an Elasticsearch document within seconds while a scheduled poller heals anything the event path missed (Bytes 27–28, 33–35), and admins search the projection with full-text, filters, and revenue aggregations (Bytes 25–26, 52) while order truth always stays in PostgreSQL (Bytes 40, 53). Catalog managers edit MongoDB freely knowing history is frozen (Bytes 24, 31, 54), the whole world seeds itself on first boot (Bytes 30–31), and every layer — isolated by names, ports, volumes, and network (Bytes 2, 58–59) — is covered by fast, Docker-free tests plus three documented gotchas (Bytes 64–70).